

# JARVIS

*Just A Rather Very Intelligent System*

## Product Vision Statement

Document de cadrage stratégique — Phase 0

|          |                                         |
|----------|-----------------------------------------|
| Version  | 1.0 — Document initial                  |
| Date     | Mai 2026                                |
| Statut   | En cours de validation                  |
| Dossier  | 01_Product_Vision                       |
| Auteur   | Équipe Projet JARVIS                    |
| Audience | Fondateurs, développeurs, investisseurs |

# 1. Résumé exécutif

*"JARVIS est un assistant IA personnel à mémoire persistante, capable de comprendre le contexte, orchestrer des agents spécialisés et agir dans des systèmes réels — pour transformer radicalement la productivité individuelle."*

Ce document établit la vision stratégique du projet JARVIS : sa raison d'être, le problème qu'il résout, les utilisateurs qu'il cible, et la direction produit qui guidera toutes les décisions d'architecture, de développement et de priorisation.

JARVIS n'est pas un chatbot avancé. C'est un système d'orchestration IA, pensé comme un vrai produit technologique — architecturé, documenté et conçu pour évoluer vers une solution scalable et crédible.

## 2. Le problème réellement résolu

### 2.1 Constat de départ

Les outils d'IA actuels — aussi puissants soient-ils — ont un défaut structurel majeur : ils sont apatrides et isolés. Chaque session repart de zéro. Chaque outil parle dans son coin. L'utilisateur reste le pivot humain de tout le système, obligé de tout orchestrer manuellement : copier-coller des contextes, répéter les mêmes briefs, basculer entre dix applications qui ne se parlent pas.

Ce n'est pas un problème d'IA. C'est un problème d'architecture. Les modèles sont capables. Ce qui manque, c'est le système qui les relie, les alimente en contexte, les coordonne et agit à la place de l'utilisateur.

### 2.2 Les quatre douleurs prioritaires

| Douleur        | Description                                   | Impact utilisateur                      |
|----------------|-----------------------------------------------|-----------------------------------------|
| Fragmentation  | Trop d'outils isolés, aucune mémoire partagée | Perte de temps, contexte dispersé       |
| Amnésie des IA | Chaque session IA recommence à zéro           | Briefs répétitifs, friction quotidienne |

|                        |                                             |                                   |
|------------------------|---------------------------------------------|-----------------------------------|
| Charge cognitive       | L'utilisateur pilote tout manuellement      | Épuisement décisionnel, erreurs   |
| Automatisation absente | Pas de workflows déclenchés automatiquement | Actions répétitives non déléguées |

### 3. Vision produit

#### 3.1 Déclaration de vision

**Vision à 3 ans**

JARVIS est le premier système d'assistant IA personnel qui centralise le contexte de l'utilisateur, maintient une mémoire persistante cross-sessions, délègue des tâches à des agents spécialisés et interagit avec des systèmes tiers — créant l'expérience d'avoir un collaborateur toujours disponible, parfaitement informé et capable d'agir.

#### 3.2 Proposition de valeur

JARVIS apporte trois promesses distinctes, cumulatives et différenciantes :

- Il se souvient. Contrairement à tout chatbot IA, JARVIS maintient une mémoire persistante de l'utilisateur, de ses projets, de ses préférences et de son historique. Chaque interaction bénéficie du contexte complet.
- Il comprend. JARVIS ne réagit pas aux mots : il comprend l'intention, anticipe le besoin suivant et adapte sa réponse au profil de l'utilisateur. Ce n'est pas un outil, c'est un collaborateur.
- Il agit. JARVIS ne se contente pas de générer du texte. Il peut déclencher des actions réelles : envoyer un email, créer une tâche, appeler une API, exécuter un workflow, mettre à jour un document.

#### 3.3 Positionnement

JARVIS se positionne à l'intersection de trois catégories aujourd'hui séparées : les assistants IA conversationnels (ChatGPT, Claude), les outils d'automatisation (Zapier, Make) et les systèmes de gestion de l'information personnelle (Notion, Obsidian). Aucun acteur actuel n'occupe ce créneau de façon cohérente.

| Dimension | Chatbot IA classique | Outil d'automatisation | JARVIS |
|-----------|----------------------|------------------------|--------|
|-----------|----------------------|------------------------|--------|

|               |                      |                      |                          |
|---------------|----------------------|----------------------|--------------------------|
| Mémoire       | Aucune (session)     | Partielle (triggers) | Persistante et profonde  |
| Compréhension | Contextuelle limitée | Aucune               | Profilée et évolutive    |
| Action réelle | Non                  | Oui (règles fixes)   | Oui (intention libre)    |
| Proactivité   | Non                  | Conditionnelle       | Oui (patterns appris)    |
| Orchestration | Non                  | Non                  | Multi-agents spécialisés |

## 4. Utilisateur cible

---

### 4.1 Profil MVP — Persona principal

**Persona : Le professionnel indépendant à haute cadence**

Fondateur, product manager, consultant ou développeur technique, travaillant seul ou en très petite équipe. Il jongle entre plusieurs projets simultanément, utilise déjà des outils IA au quotidien, mais est frustré par leur manque de mémoire et d'autonomie. Il cherche un levier de productivité réel, pas une démo. Il est prêt à adopter un outil imparfait si le gain de temps est immédiatement perceptible.

### 4.2 Caractéristiques du persona

- Utilise activement Claude, ChatGPT ou Perplexity au quotidien
- Gère 3 à 8 projets simultanément avec des contextes très différents
- Souffre du temps perdu à re-contextualiser chaque outil à chaque session
- Cherche à automatiser les tâches répétitives sans compétences DevOps
- Valorise l'efficacité, la fiabilité et la transparence du système

### 4.3 Utilisateurs secondaires (V2+)

- Équipes de 2 à 10 personnes souhaitant un assistant partagé avec mémoire collective
- Développeurs souhaitant intégrer JARVIS comme couche d'intelligence dans leurs propres outils
- Entreprises cherchant une solution d'assistant IA on-premise avec contrôle des données

## 5. Cas d'usage prioritaires

---

5.1 Cas d'usage MVP — Phase 1

| Cas d'usage          | Description                                                   | Valeur clé                     | Priorité |
|----------------------|---------------------------------------------------------------|--------------------------------|----------|
| Chat contextuel      | Conversation avec mémoire persistante du profil utilisateur   | Éliminer les briefs répétitifs | Critique |
| Gestion de tâches    | Capture et priorisation de tâches depuis la conversation      | Centralisation productive      | Critique |
| Mémoire personnelle  | Stockage et rappel d'informations clés sur l'utilisateur      | Personnalisation profonde      | Critique |
| Recherche & synthèse | Recherche web et résumé de documents uploadés                 | Gain de temps intellectuel     | Élevée   |
| Dashboard personnel  | Vue centralisée des tâches, mémoires et interactions récentes | Visibilité et contrôle         | Élevée   |

5.2 Cas d'usage V2 — Phase 2

- Automatisation d'actions : envoi d'emails, création de documents, appels API sur commande vocale ou textuelle
- Intégrations tierces : connexion à Gmail, Google Calendar, Notion, GitHub, Slack
- Voice input : compréhension vocale pour les commandes et les requêtes
- Workflows déclenchés : automatisations conditionnelles basées sur des événements

5.3 Cas d'usage V3 — Phase 3

- Orchestration multi-agents : délégation à des agents spécialisés (Finance, Code, Planning, Research)
- Proactivité système : JARVIS anticipe et agit sans être sollicité sur des patterns récurrents
- Mémoire collective : partage de contexte entre plusieurs utilisateurs d'une même équipe
- Plugins tiers : API publique permettant d'étendre JARVIS avec des agents custom

6. Fonctionnalités MVP — Scope exact

6.1 Ce qui est inclus dans le MVP

| Fonctionnalité    | Description technique                                    | Statut |
|-------------------|----------------------------------------------------------|--------|
| Interface de chat | Interface web responsive avec historique de conversation | MVP    |
| Authentification  | Auth sécurisée (JWT + OAuth), multi-utilisateurs         | MVP    |

|                            |                                                         |     |
|----------------------------|---------------------------------------------------------|-----|
| <b>Mémoire court terme</b> | Contexte de session maintenu dans le prompt window      | MVP |
| <b>Mémoire long terme</b>  | Base vectorielle pgvector pour faits persistants        | MVP |
| <b>Profil utilisateur</b>  | Stockage structuré des préférences et informations clés | MVP |
| <b>Gestion de tâches</b>   | CRUD de tâches extrait de la conversation               | MVP |
| <b>Recherche web</b>       | Intégration API de recherche (Serper / Tavily)          | MVP |
| <b>Upload de documents</b> | Analyse et résumé de PDF et fichiers texte              | MVP |
| <b>Dashboard</b>           | Vue synthétique des tâches, mémoires, interactions      | MVP |
| <b>LLM principal</b>       | Claude API (Anthropic) avec fallback OpenAI             | MVP |

## 6.2 Ce qui est explicitement exclu du MVP

Les éléments suivants sont écartés du MVP non par manque d'intérêt, mais parce qu'ils introduisent une complexité disproportionnée au regard de la valeur délivrée à ce stade.

- Voice input / output : latence, complexité STT/TTS et coût infra non justifiés avant validation du concept core
- Application mobile native : effort frontend trop élevé pour la phase de validation
- Orchestration multi-agents : architecturalement prématurée avant stabilisation du système mémoire
- Intégrations tierces multiples : une seule intégration (Gmail ou Notion) suffit pour valider la connectivité
- Mode temps réel / streaming d'événements : WebSockets et infrastructure event-driven à reporter en V2

## 7. Architecture technique — Orientations stratégiques

### 7.1 Principes architecturaux

L'architecture de JARVIS doit respecter quatre principes non négociables dès la phase MVP :

- Modularité : chaque composant (mémoire, agents, APIs) est isolé et remplaçable sans refactoring global

- Évolutivité : l'architecture monolithique du MVP doit permettre une extraction progressive vers des microservices ou agents sans rupture
- Contrôle des coûts : chaque appel LLM est mesuré, optimisé et auditable — la compression de contexte est implémentée dès le départ
- Sécurité by design : les données de mémoire personnelle sont chiffrées, isolées par utilisateur et auditées

7.2 Stack technique envisagée

| Couche              | Technologies retenues     | Justification                                 |
|---------------------|---------------------------|-----------------------------------------------|
| LLM Principal       | Claude API (Anthropic)    | Qualité de raisonnement, context window large |
| LLM Fallback        | OpenAI GPT-4o             | Résilience et complémentarité                 |
| Orchestration IA    | LangChain / LangGraph     | Framework mature, agents et RAG intégrés      |
| Backend             | FastAPI (Python)          | Performance, async natif, typage fort         |
| Mémoire vectorielle | PostgreSQL + pgvector     | Pragmatique, évolutif, un seul système de BDD |
| Cache & sessions    | Redis                     | Mémoire court terme ultra-rapide              |
| Automatisations     | n8n (self-hosted)         | Workflows visuels, 400+ intégrations natives  |
| Frontend            | Next.js + TailwindCSS     | SSR, performance, DX excellente               |
| Infrastructure      | Docker + Railway / Fly.io | Déploiement rapide, scalable                  |

8. Roadmap — Vue synthétique

| Phase         | Durée         | Objectifs clés                                                           | Livrable principal            |
|---------------|---------------|--------------------------------------------------------------------------|-------------------------------|
| Phase 0       | 2 semaines    | Cadrage, architecture, documentation initiale, choix de stack définitifs | 8 documents fondateurs        |
| Phase 1 — MVP | 6–8 semaines  | Chat + mémoire + tâches + interface + auth + déploiement                 | JARVIS MVP fonctionnel        |
| Phase 2 — V2  | 6–10 semaines | Automatisations, intégrations API, voice input, dashboard avancé         | JARVIS connecté et automatisé |
| Phase 3 — V3  | Long terme    | Multi-agents, proactivité, mémoire collective, API publique              | JARVIS plateforme             |

8.1 Critères de passage Phase 0 → Phase 1

- Vision produit documentée et validée (ce document)
- Architecture système définie et documentée (System Architecture Overview)
- Stack technique décidée et justifiée (Tech Stack Decision Log)
- Système mémoire architecturé (Memory System Design)
- Scope MVP figé, backlog Phase 1 priorisé

9. Risques et contraintes majeures

| Risque               | Niveau   | Description                                                      | Mitigation                                                   |
|----------------------|----------|------------------------------------------------------------------|--------------------------------------------------------------|
| Scope creep          | Critique | Tentation d'ajouter des features avant de valider les fondations | Scope MVP figé et non négociable jusqu'à livraison           |
| Architecture mémoire | Élevé    | Mémoire mal conçue → latence, incohérence, coûts explosifs       | Design dédié avant tout développement (Memory System Design) |
| Coûts LLM            | Élevé    | Tokens consommés sur contextes longs non compressés              | Stratégie de compression et token budget dès le MVP          |
| Complexité voice     | Moyen    | STT/TTS ajoute latence, coût et surface de bugs                  | Reporté explicitement en V2                                  |
| Sécurité données     | Moyen    | Mémoire personnelle sensible mal protégée                        | Auth robuste et chiffrement dès le MVP                       |
| Dépendance LLM       | Moyen    | Changements d'API ou de tarification Anthropic/OpenAI            | Abstraction LLM dans le code — swap transparent              |

10. Structure documentaire du projet

Toute la documentation JARVIS est organisée dans une structure cohérente inspirée des standards startup IA/SaaS. Chaque document est pensé pour être directement exploitable par des développeurs, des product managers et des investisseurs.

| Dossier           | Document                   | Objectif                                                     |
|-------------------|----------------------------|--------------------------------------------------------------|
| 01_Product_Vision | Product Vision Statement ✓ | Ce document — vision, positionnement, proposition de valeur  |
| 02_MVP            | MVP Feature Spec           | Scope exact du MVP, critères de done, exclusions documentées |
| 03_User_Flows     | User Flow Diagrams         | Parcours utilisateurs clés pour les 5 cas d'usage MVP        |



|                        |                                |                                                              |
|------------------------|--------------------------------|--------------------------------------------------------------|
| 04_System_Architecture | System Architecture Overview   | Schéma global des composants, flux de données, interfaces    |
| 05_AI_Architecture     | AI Architecture & LLM Strategy | Choix du modèle, prompting system, RAG, gestion contexte     |
| 06_Agent_System        | Agent System Design v1         | Orchestrateur, agents spécialisés, protocoles                |
| 12_Memory_System       | Memory System Design           | Architecture mémoire court/long terme, vectorDB, compression |
| 16_Roadmap             | Product Roadmap v1             | Phases, jalons, dépendances, critères de passage             |
| 20_Dev_Log             | Tech Stack Decision Log        | Choix techniques, justifications, alternatives écartées      |

